

Année universitaire 2025/2026

Rapport de projet de fin d'année

3^e année

Ingénierie Informatique et Réseaux

Sous le thème

Système de gestion d'agence bancaire

Application web AmanaBank

Réalisé par :

BENSOUDA Amina

AFILAL Soraya

Encadrée par :

Dr. ORCHI HOUDA

École Marocaine des Sciences de l'Ingénieur de Rabat

Dédicaces

Nous dédions ce projet de fin d'année à nos parents, dont le soutien inébranlable, l'amour et les encouragements constants ont été des piliers essentiels tout au long de notre parcours académique. Leur confiance en nos capacités nous a toujours inspirés à donner le meilleur de nous-mêmes.

À Dr. ORCHI HOUDA, notre encadrante, pour ses conseils avisés, sa disponibilité et son expertise. Merci pour votre soutien inestimable et votre guidance précieuse tout au long de ce projet.

Nous souhaitons également exprimer notre profonde gratitude envers l'ensemble du corps professoral de l'École Marocaine des Sciences de l'Ingénieur pour la qualité de la formation reçue et l'exigence qui nous a permis de développer des compétences techniques et une rigueur professionnelle.

Enfin, nous dédions ce travail à toutes les personnes qui, de près ou de loin, ont contribué à notre développement personnel et académique.

Remerciements

Nous tenons tout d'abord à remercier chaleureusement Dr. ORCHI HOUDA, pour sa guidance experte, son soutien constant et ses précieux conseils tout au long de ce projet. Son expertise et son enthousiasme ont été une source d'inspiration et ont grandement enrichi notre expérience académique.

Nous remercions cordialement tout le corps professoral et administratif de l'École Marocaine des Sciences de l'Ingénieur pour son accompagnement tout au long de notre formation en Ingénierie Informatique et Réseaux.

Nous sommes reconnaissantes envers nos familles et nos proches pour leur soutien inconditionnel et leur compréhension durant cette période intensive. Leur encouragement a été le moteur de notre persévérance.

Enfin, merci à toutes les personnes qui ont participé, de près ou de loin, à l'élaboration de ce projet.

À toutes et à tous... merci.

Résumé

La gestion quotidienne d'une agence bancaire implique le traitement de données clients sensibles, la tenue rigoureuse des comptes et l'exécution d'opérations financières soumises à des règles métier strictes. Les méthodes traditionnelles, souvent manuelles ou dispersées, augmentent les risques d'erreurs de saisie, d'incohérences de solde et de faible traçabilité.

Face à ces limites, ce projet propose **AmanaBank**, une application web centralisée développée avec **Java** et **Spring Boot** par BENSOU DA Amina et AFILAL Soraya dans le cadre d'un projet de fin d'année à l'École Marocaine des Sciences de l'Ingénieur.

La solution couvre la gestion interne de l'agence (clients, comptes, transactions, utilisateurs, audit, reporting) ainsi que des extensions : paiement de factures, commande de chéquier, centre de notifications et portail client en ligne. Chaque acteur dispose d'un espace adapté à son rôle.

Le présent rapport présente le contexte du projet, l'analyse des besoins, la conception UML, l'environnement technique, l'architecture logicielle, la réalisation des interfaces et les perspectives d'évolution.

Mots-clés : Agence bancaire — Spring Boot — Gestion de comptes — Transactions — Portail client — UML

Abstract

Daily operations in a bank branch involve sensitive customer data, strict account management, and financial transactions governed by business rules. Traditional, often manual approaches increase the risk of data entry errors, balance inconsistencies, and poor traceability.

To address these challenges, this project introduces **AmanaBank**, a centralized web application built with **Java** and **Spring Boot** by BENSOUA Amina and AFILAL Soraya as part of a final-year project at EMSI Rabat.

The solution supports internal branch management (clients, accounts, transactions, users, audit, reporting) as well as extended features : bill payments, checkbook orders, an in-app notification center, and an online client portal. Each stakeholder has a role-based workspace.

This report presents the project context, requirements analysis, UML design, technical environment, software architecture, interface implementation, and future perspectives.

Keywords : Bank branch — Spring Boot — Account management — Transactions — Client portal — UML

Table des matières

Dédicaces	iii
Remerciements	v
Résumé	vii
Abstract	ix
Liste des abréviations	xiii
Introduction générale	1
0.1 Contexte général	1
0.2 Problématique	1
0.3 Objectifs du projet	1

0.4	Plan du rapport	2
1	Contexte du projet et méthodologie	3
1.1	Introduction	3
1.2	Présentation du contexte et de l'organisme d'accueil	3
1.2.1	Contexte académique	3
1.2.2	Contexte métier	3
1.2.3	Équipe projet	3
1.3	Étude de l'existant et critique	4
1.3.1	Description de l'existant	4
1.3.2	Critique de l'existant	4
1.3.3	Solution proposée	4
1.4	Méthodologie de travail	4
1.4.1	Processus de développement	4
1.4.2	Planification	5
1.5	Conclusion	5
2	Analyse des besoins et conception	7
2.1	Introduction	7
2.2	Identification des acteurs	7
2.3	Analyse des besoins fonctionnels	7
2.3.1	Périmètre	7
2.3.2	Fonctionnalités principales	8
2.3.3	Règles de gestion	8
2.4	Modélisation UML	8
2.4.1	Diagrammes de cas d'utilisation	8
2.4.2	Diagramme de classes	9
2.4.3	Modèle de données (MCD et MLD)	10
2.4.4	Diagrammes de séquence	10
2.4.5	Diagramme d'activité	11
2.5	Conclusion	11
3	Environnement de travail et technologies	13
3.1	Introduction	13
3.2	Environnement de travail	13
3.2.1	Poste de développement	13
3.2.2	Base de données	13
3.2.3	Outils de gestion de version	13
3.3	Technologies et outils utilisés	14
3.3.1	Back-end	14

3.3.2	Front-end	14
3.3.3	Outils de conception et documentation	14
3.3.4	Choix technologiques	14
3.4	Conclusion	15
4	Architecture technique de l'application	17
4.1	Introduction	17
4.2	Architecture globale	17
4.3	Arborescence du back-end	17
4.3.1	Point d'entrée et configuration	18
4.3.2	Couche domaine	18
4.3.3	Couche persistance	18
4.3.4	Couche métier (services)	19
4.3.5	Couche web (contrôleurs)	19
4.3.6	Sécurité	19
4.4	Front-end (Thymeleaf)	20
4.4.1	Structure des templates	20
4.4.2	Ressources statiques	20
4.4.3	Composants transverses	20
4.5	Conclusion	20
5	Réalisation et implémentation des fonctionnalités	21
5.1	Introduction	21
5.2	Espace public et connexion	21
5.2.1	Page d'accueil (landing)	21
5.2.2	Page de connexion	22
5.3	Espace agence — personnel	22
5.3.1	Tableau de bord	22
5.3.2	Gestion des clients	22
5.3.3	Comptes bancaires	23
5.3.4	Opérations financières	23
5.3.5	Historique et reçu PDF	24
5.3.6	Commandes de chéquier et notifications	24
5.3.7	Administration	25
5.4	Espace client (portail)	25
5.5	Conclusion	25
	Conclusion générale	27
	Bibliographie	29

Table des figures

2.1	Cas d'utilisation — clients et comptes	8
2.2	Cas d'utilisation — opérations et supervision	9
2.3	Cas d'utilisation — portail client	9
2.4	Diagramme de classes	9
2.5	Modèle conceptuel de données (MCD)	10
2.6	Modèle logique de données (MLD)	10
2.7	Séquence — authentification dual (personnel / client)	10
2.8	Séquence — opérations financières	10
2.9	Séquence — paiement de facture	11
2.10	Séquence — commande de chéquier	11
2.11	Activité — virement bancaire	11
5.1	Page d'accueil publique AmanaBank	21
5.2	Connexion personnel et client (CIN, n° client ou username)	22
5.3	Dashboard avec indicateurs et alertes chéquier	22
5.4	Liste et recherche des clients	22
5.5	Fiche détail compte — solde, statut, actions	23
5.6	Formulaire de dépôt	23
5.7	Paiement de facture (LYDEC, ONEE, IAM...)	23
5.8	Reçu d'opération avec téléchargement PDF	24
5.9	Suivi des commandes de chéquier	24
5.10	Centre de notifications (chef d'agence)	24
5.11	Gestion des utilisateurs internes	25
5.12	Tableau de bord espace client	25
5.13	Notifications client après opération en agence	25

Liste des tableaux

1.1	Phases principales du projet AmanaBank	5
2.1	Acteurs du système AmanaBank	7
2.2	Modules fonctionnels	8
3.1	Stack back-end	14
3.2	Stack front-end	14
4.1	Packages de configuration	18
4.2	Entités et énumérations	18
4.3	Services principaux	19
4.4	Organisation des templates	20

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
BCrypt	Algorithme de hachage de mots de passe
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JWT	JSON Web Token
KPI	Key Performance Indicator
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PDF	Portable Document Format
PFA	Projet de Fin d'Année
SQL	Structured Query Language
UML	Unified Modeling Language

Introduction générale

0.1 Contexte général

La digitalisation des services bancaires constitue aujourd’hui un enjeu majeur pour les établissements financiers. Au sein d’une agence, les agents traitent quotidiennement des demandes variées : ouverture de comptes, dépôts, retraits, virements, consultation d’historiques et supervision des opérations. Sans outil centralisé, ces tâches reposent souvent sur des registres papier, des fichiers tableur ou des logiciels non intégrés, ce qui complique le contrôle des soldes et la traçabilité des actions.

Dans le cadre de notre formation en Ingénierie Informatique et Réseaux à l’École Marocaine des Sciences de l’Ingénieur, nous avons développé **AmanaBank**, un système web de gestion d’agence bancaire répondant au cahier des charges du projet de fin d’année (PFA). Ce travail s’inscrit dans une démarche de conception rigoureuse (UML, modèle de données) suivie d’une implémentation incrémentale en Java.

0.2 Problématique

Comment concevoir et réaliser une application web fiable permettant à une agence bancaire de :

- centraliser la gestion des clients et de leurs comptes ;
- exécuter des opérations financières en respectant les règles métier (solde, statut des comptes) ;
- assurer la traçabilité des actions sensibles et la sécurité des accès ;
- offrir au client un espace de consultation et de suivi en ligne ?

Les approches manuelles ne garantissent ni la cohérence des soldes, ni l’identification de l’agent exécutant, ni une supervision efficace pour le chef d’agence.

0.3 Objectifs du projet

Les objectifs principaux sont :

1. **Analyser** le besoin à partir du cahier des charges et modéliser le système (UML, MCD/MLD).
2. **Concevoir** une architecture en couches (Spring Boot, PostgreSQL, Flyway).

3. **Réaliser** les fonctionnalités métier : clients, comptes, transactions, utilisateurs, reporting.
4. **Étendre** le périmètre : paiement de factures, commande de chéquier, notifications, portail client.
5. **Valider** le système par des tests d'intégration et une démonstration reproductible.

0.4 Plan du rapport

Ce rapport est structuré en cinq chapitres :

- Le **chapitre 1** présente le contexte du projet, l'organisme d'accueil, l'étude de l'existant et la méthodologie de travail.
- Le **chapitre 2** détaille l'analyse des besoins, l'identification des acteurs et la modélisation UML.
- Le **chapitre 3** décrit l'environnement de travail et les technologies utilisées.
- Le **chapitre 4** expose l'architecture technique du back-end et du front-end.
- Le **chapitre 5** illustre la réalisation par les captures d'écran des interfaces.

Une **conclusion générale** synthétise le bilan, les difficultés rencontrées et les perspectives d'évolution.

Chapitre 1

Contexte du projet et méthodologie

1.1 Introduction

Ce chapitre situe notre projet dans son contexte institutionnel et métier. Nous présentons l'organisme d'accueil, l'étude de l'existant avec sa critique, puis la méthodologie adoptée pour mener le développement.

1.2 Présentation du contexte et de l'organisme d'accueil

1.2.1 Contexte académique

Ce projet a été réalisé au sein de l'**École Marocaine des Sciences de l'Ingénieur (de Rabat)**, dans le cadre du **projet de fin d'année (PFA)** de 3^e année en Ingénierie Informatique et Réseaux. Il répond au cahier des charges fourni par l'établissement et a été encadré par Dr. ORCHI HOUDA.

1.2.2 Contexte métier

Le projet simule la gestion d'une **agence bancaire fictive** baptisée **AmanaBank**. L'objectif est de reproduire les processus quotidiens d'une agence : accueil client, ouverture de comptes, opérations financières, supervision et reporting. L'organisme d'accueil académique nous a fourni le cadre méthodologique ; le domaine métier s'inspire des pratiques bancaires courantes au Maroc.

1.2.3 Équipe projet

Le projet a été mené en binôme :

- **BENSOUDA Amina**
- **AFILAL Soraya**

1.3 Étude de l'existant et critique

1.3.1 Description de l'existant

Dans de nombreuses agences de taille modeste, la gestion repose encore sur :

- Des fiches clients papier ou des fichiers non centralisés.
- Un suivi des soldes via tableur, sans contrôle automatique des règles métier.
- Des opérations saisies manuellement, sujettes aux erreurs humaines.
- L'absence de journal centralisé des actions sensibles.
- Aucun accès en ligne pour le client.

1.3.2 Critique de l'existant

- **Manque d'efficacité** : saisies répétitives, recherches longues, doublons possibles (CIN).
- **Incohérences de solde** : absence de verrouillage transactionnel.
- **Faible traçabilité** : difficulté à identifier l'agent responsable d'une opération.
- **Sécurité insuffisante** : mots de passe ou données sans politique homogène.
- **Supervision limitée** : pas de tableau de bord ni d'alertes pour le chef d'agence.

1.3.3 Solution proposée

Nous proposons **AmanaBank**, une application web intégrée couvrant :

- Gestion des **clients** et **comptes** (ouverture, blocage, clôture).
- **Opérations financières** : dépôt, retrait, virement, paiement de factures.
- **Administration** du personnel et journal d'**audit**.
- **Reporting** : dashboard, relevés, reçus HTML et PDF.
- **Extensions** : chéquier, notifications, portail client.

1.4 Méthodologie de travail

1.4.1 Processus de développement

Nous avons adopté une approche **itérative et incrémentale**, inspirée du processus **2TUP** (Two-Track Unified Process) :

1. **Piste conception** : analyse du cahier des charges, diagrammes UML, MCD/MLD.
2. **Piste réalisation** : développement par phases verticales, chacune livrant une fonctionnalité testable.

Le projet a été découpé en **quatre pistes logiques** (voir tableau 1.1) :

- **Noyau** (phases 1 à 8) : cahier des charges.
- **Extensions agence** (phases 10 et 11) : factures, chéquier.
- **Canal client** (phases 13 et 14) : notifications, portail.
- **Clôture** (phase 12) : démo, documentation, soutenance.

1.4.2 Planification

TABLE 1.1 – Phases principales du projet AmanaBank

Phase	Intitulé	Livrable
1	Conception UML	Cas d'utilisation, classes, séquences, MCD/MLD
2–3	Bootstrap + Auth	Projet Spring Boot, login, rôles
4–6	Métier cœur	Clients, comptes, transactions
7–8	Admin + Reporting	Utilisateurs, dashboard, audit
10–11	Extensions	Palement facture, chéquier
13–14	Canal client	Notifications, portail
12	Clôture	Manuel, script démo, rapport

Chaque phase a été validée par des tests (`mvn test`) avant de passer à la suivante.

1.5 Conclusion

Ce chapitre a situé le projet dans son contexte académique et métier, mis en évidence les limites des approches manuelles et présenté notre méthodologie itérative. Le chapitre suivant détaille l'analyse des besoins et la conception du système.

Chapitre 2

Analyse des besoins et conception

2.1 Introduction

Ce chapitre présente l'analyse des besoins fonctionnels de AmanaBank, l'identification des acteurs et la modélisation UML qui a guidé l'implémentation.

2.2 Identification des acteurs

Le système distingue **quatre acteurs principaux** :

TABLE 2.1 – Acteurs du système AmanaBank

Acteur	Rôle et responsabilités
Agent bancaire	Clients, comptes, opérations courantes, historique, chèquiers
Chef d'agence	Supervision, dashboard, notifications, journal d'audit
Administrateur	Gestion du personnel (création, rôles, activation)
Client bancaire	Portail en ligne : consultation, reçus, notifications

L'authentification est unifiée sur `/login` ; la redirection post-connexion dépend du rôle (personnel $\rightarrow$ `/dashboard`, client $\rightarrow$ `/portal/dashboard`).

2.3 Analyse des besoins fonctionnels

2.3.1 Périmètre

- **Inclus** : CRUD clients/comptes, opérations financières, utilisateurs, audit, reporting, paiement facture, chéquier, notifications, portail client.
- **Exclu** : crédit, cartes bancaires, GAB, interbancaire, microservices.

2.3.2 Fonctionnalités principales

TABLE 2.2 – Modules fonctionnels

Module	Description
Authentification	Login, BCrypt, rôles ADMIN / AGENT / CHEF_AGENCE / CLIENT
Clients	CRUD, recherche multicritère, statuts
Comptes	Ouverture, blocage, clôture (solde nul requis)
Opérations	Dépôt, retrait, virement avec règles métier
Paieement facture	LYDEC, ONEE, IAM... — débit compte + reçu
Chéquier	Workflow PENDING → DELIVERED, sans impact solde
Reporting	Dashboard, relevé, reçu HTML + PDF (OpenPDF)
Notifications	Cloche, alertes chéquier, notifications client
Portail	Consultation comptes, historique, chèquiers

2.3.3 Règles de gestion

- R1–R2 : un client possède un ou plusieurs comptes ; un compte appartient à un seul client.
- R3 : retrait/paieement refusé si solde insuffisant.
- R4 : virement — comptes source et destination actifs.
- R5–R8 : traçabilité, comptes bloqués/clôturés inopérants, BCrypt, audit.
- R9–R14 : extensions facture et chéquier (référence obligatoire, une commande PENDING max).

2.4 Modélisation UML

2.4.1 Diagrammes de cas d'utilisation

*[Cas d'utilisation — clients et comptes — fichier attendu :
figures/uml/01-clients-comptes.pdf]*

FIGURE 2.1 – Cas d'utilisation — clients et comptes

*[Cas d'utilisation — opérations et supervision — fichier attendu :
figures/uml/02-operations-supervision.pdf]*

FIGURE 2.2 – Cas d'utilisation — opérations et supervision

*[Cas d'utilisation — portail client — fichier attendu :
figures/uml/03-portail-client.pdf]*

FIGURE 2.3 – Cas d'utilisation — portail client

2.4.2 Diagramme de classes

*[Diagramme de classes — fichier attendu :
figures/uml/diagramme-classes.pdf]*

FIGURE 2.4 – Diagramme de classes

Les entités principales sont : User, Client, Account, Transaction, BillProvider, BillPayment, CheckbookOrder, Notification, AuditLog.

2.4.3 Modèle de données (MCD et MLD)

*[Modèle conceptuel de données (MCD) — fichier attendu :
figures/uml/MCD.pdf]*

FIGURE 2.5 – Modèle conceptuel de données (MCD)

[Modèle logique de données (MLD) — fichier attendu : figures/uml/MLD.pdf]

FIGURE 2.6 – Modèle logique de données (MLD)

2.4.4 Diagrammes de séquence

*[Séquence — authentication dual (personnel / client) — fichier attendu :
figures/uml/01-authentication.pdf]*

FIGURE 2.7 – Séquence — authentication dual (personnel / client)

*[Séquence — opérations financières — fichier attendu :
figures/uml/03-operations-financieres.pdf]*

FIGURE 2.8 – Séquence — opérations financières

*[Séquence — paiement de facture — fichier attendu :
figures/uml/06-paiement-facture.pdf]*

FIGURE 2.9 – Séquence — paiement de facture

*[Séquence — commande de chéquier — fichier attendu :
figures/uml/07-commande-chequier.pdf]*

FIGURE 2.10 – Séquence — commande de chéquier

2.4.5 Diagramme d'activité

*[Activité — virement bancaire — fichier attendu :
figures/uml/diagramme-activite-virement.pdf]*

FIGURE 2.11 – Activité — virement bancaire

2.5 Conclusion

Ce chapitre a identifié les acteurs, formalisé les besoins fonctionnels et présenté la modélisation UML complète. Le chapitre suivant détaille l'environnement technique et les outils de développement.

Chapitre 3

Environnement de travail et technologies

3.1 Introduction

Ce chapitre présente l'environnement matériel et logiciel utilisé pour le développement de AmanaBank, ainsi que les technologies et frameworks retenus.

3.2 Environnement de travail

3.2.1 Poste de développement

- Système d'exploitation : Windows 10/11
- Processeur et mémoire adaptés au développement Java et PostgreSQL
- IDE : Visual Studio Code / Cursor (édition) + terminal Maven

3.2.2 Base de données

- **PostgreSQL 18** installé localement (port **5433**)
- Base : `banque_agence`, utilisateur `banque`
- Script d'initialisation : `documentation/setup-postgresql.sql`

3.2.3 Outils de gestion de version

- **Git** pour le versionnement du code source
- Dépôt structuré par phases (commits incrémentaux)

3.3 Technologies et outils utilisés

3.3.1 Back-end

TABLE 3.1 – Stack back-end

Technologie	Rôle
Java 21	Langage principal
Spring Boot 3.4	Framework applicatif (web, sécurité, JPA)
Spring Security	Authentification, autorisation par rôles
Spring Data JPA	Persistance objet-relationnelle (Hibernate)
Flyway	Migrations versionnées de la base (V1 à V11)
Bean Validation	Validation des formulaires côté serveur
OpenPDF 2.0.3	Génération des reçus PDF
Maven 3.9+	Build, dépendances, tests

3.3.2 Front-end

TABLE 3.2 – Stack front-end

Technologie	Rôle
Thymeleaf	Moteur de templates serveur (MVC)
Bootstrap 5.3	Framework CSS responsive
Bootstrap Icons	Icônes de navigation (sidebar, cloche)
CSS personnalisé	Charte graphique AmanaBank (marron/beige/or)
JavaScript	Scripts légers (landing page)

3.3.3 Outils de conception et documentation

- **PlantUML** : diagrammes UML (cas d'utilisation, classes, séquences, activité, MCD/MLD).
- **StarUML** ou éditeur texte : alternative pour la modélisation.
- **GanttProject** : planification des phases (optionnel).
- **JUnit 5 + Spring Boot Test + MockMvc** : tests d'intégration (80 tests).

3.3.4 Choix technologiques

Nous avons retenu un **monolithe Spring Boot** pour les raisons suivantes :

- Adapté à un projet PFA solo/binôme — un seul dépôt, un JAR déployable.
- Spring Security offre une story solide pour l'authentification bancaire.
- Thymeleaf évite la complexité d'un SPA React/Angular séparé.

- PostgreSQL garantit l'intégrité relationnelle des données financières.
- Flyway permet une évolution reproductible du schéma (défendable en soutenance).

3.4 Conclusion

Ce chapitre a présenté l'environnement de développement et la stack technique de AmanaBank. Le chapitre suivant détaille l'architecture logicielle et l'organisation du code source.

Chapitre 4

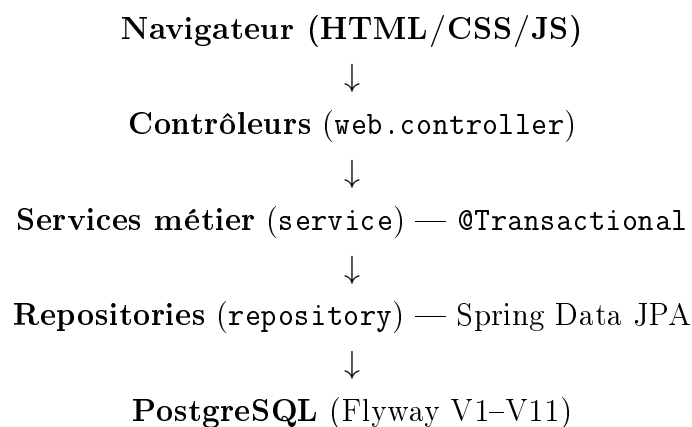
Architecture technique de l'application

4.1 Introduction

Ce chapitre décrit l'architecture logicielle de AmanaBank : organisation du back-end Java, structure du front-end Thymeleaf, et flux de données entre les couches.

4.2 Architecture globale

L'application suit une **architecture en couches** (layered monolith) :



Transversal : **Spring Security** (filtre HTTP), **AuditService**, **NotificationMode-
lAdvice**.

4.3 Arborescence du back-end

Le code source se trouve dans `src/main/java/com/banque/agence/`.

4.3.1 Point d'entrée et configuration

TABLE 4.1 – Packages de configuration

Fichier / classe	Rôle
BanqueAgenceApplication.java	Point d'entrée Spring Boot
config/SecurityConfig.java	Chaîne de sécurité, rôles, redirection login
config/DevUserInitializer.java	Seed utilisateurs (admin, agent, chef)
config/DevDemoDataInitializer.java	Seed données métier (clients, comptes...)
config/DemoPortalSync.java	Activation portail client démo
application.yml	Port 8081, profil actif
application-dev.yml	Connexion PostgreSQL, flags seed

4.3.2 Couche domaine

TABLE 4.2 – Entités et énumérations

Package	Contenu
domain/entity/	User, Client, Account, Transaction, BillProvider, BillPayment, CheckbookOrder, Notification, AuditLog
domain/enums/	UserRole, AccountType, AccountStatus, TransactionType, ClientStatus, CheckbookOrderStatus, NotificationType...

4.3.3 Couche persistance

- repository/ : interfaces Spring Data JPA (ClientRepository, AccountRepository, TransactionRepository...).
- TransactionSpecifications.java : filtres dynamiques pour l'historique.
- Migrations SQL : src/main/resources/db/migration/V1__ à V11__.

4.3.4 Couche métier (services)

TABLE 4.3 – Services principaux

Service	Responsabilité
ClientService	CRUD clients, recherche, changement statut
AccountService	Ouverture, blocage, clôture comptes
TransactionService	Dépôt, retrait, virement (@Transactional)
BillPaymentService	Paieement facture + transaction liée
CheckbookOrderService	Workflow chéquier sans impact solde
DashboardService	KPIs et opérations récentes
NotificationService	Création et lecture notifications
ClientPortalService	Données portail client (lecture seule)
ReceiptPdfService	Génération PDF OpenPDF
UserService	Administration personnel
AuditService	Journalisation actions sensibles

4.3.5 Couche web (contrôleurs)

Agence (`web.controller/`) :

- HomeController — landing /
- LoginController, DashboardController
- ClientController, AccountController
- OperationController — dépôt, retrait, virement, facture
- TransactionController — historique, reçu HTML/PDF
- CheckbookOrderController, NotificationController
- AdminUserController, AuditController

Portail client (`web.controller.portal/`) :

- PortalDashboardController, PortalAccountController
- PortalTransactionController, PortalCheckbookController
- PortalNotificationController

DTO (`web.dto/`) : objets formulaire (ClientForm, OperationForm, BillPaymentForm...).

4.3.6 Sécurité

- UserDetailsServiceImpl : charge User (personnel) ou Client (portail).
- SecurityUser / SecurityClient : wrappers Spring Security.
- Routes : `/portal/**` → `ROLE_CLIENT`; `/admin/**` → `ROLE_ADMIN`.

4.4 Front-end (Thymeleaf)

4.4.1 Structure des templates

Les vues se trouvent dans `src/main/resources/templates/`.

TABLE 4.4 – Organisation des templates

Dossier / fichier	Rôle
<code>landing.html</code>	Page vitrine publique AmanaBank
<code>login.html</code>	Connexion unique personnel + client
<code>layout/main.html</code>	Layout agence (sidebar, cloche, menu)
<code>dashboard.html</code>	Tableau de bord KPIs et alertes
<code>clients/</code>	Liste, formulaire, fiche détail client
<code>accounts/</code>	Liste, ouverture, détail, relevé
<code>operations/</code>	Dépôt, retrait, virement, paiement facture
<code>transactions/</code>	Historique, reçu imprimable
<code>checkbook-orders/</code>	Liste, demande, détail workflow
<code>notifications/</code>	Centre de notifications personnel
<code>admin/users/</code>	Gestion utilisateurs internes
<code>admin/audit/</code>	Journal d'audit
<code>portal/layout.html</code>	Layout espace client
<code>portal/</code>	Dashboard, comptes, historique, chèquiers, notifications
<code>error/403.html, 404.html</code>	Pages d'erreur

4.4.2 Ressources statiques

- `static/css/style.css` — thème agence et portail (palette marron/beige/or).
- `static/css/landing.css` — styles page vitrine.
- `static/js/landing-header.js` — navigation landing.
- `static/images/` — logo AmanaBank, favicon, images hero.

4.4.3 Composants transverses

- `NotificationModelAdvice` : injecte `unreadNotificationCount` dans toutes les vues.
- `thymeleaf-extras-springsecurity6` : affichage conditionnel selon rôle (`sec:authorize`).
- Messages flash (`RedirectAttributes`) pour succès et erreurs métier.

4.5 Conclusion

Ce chapitre a présenté l'architecture en couches de AmanaBank, l'organisation du back-end Java et la structure du front-end Thymeleaf. Le chapitre suivant illustre la réalisation par les captures d'écran des interfaces.

Chapitre 5

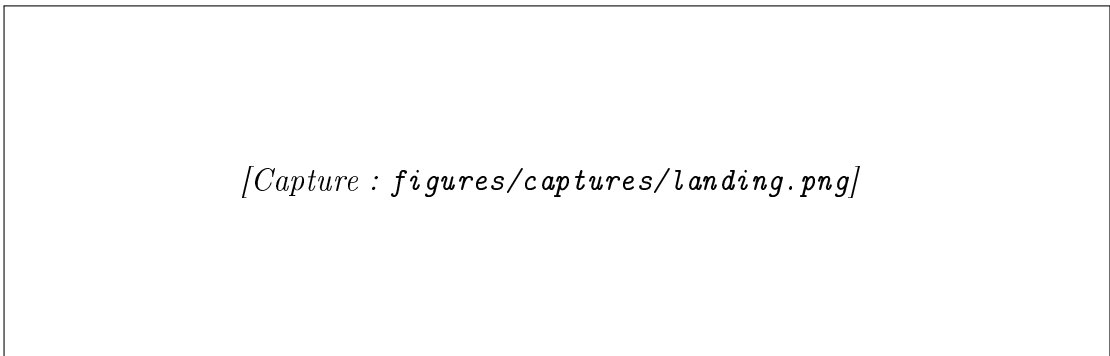
Réalisation et implémentation des fonctionnalités

5.1 Introduction

Ce chapitre présente les interfaces réalisées de AmanaBank. Chaque figure correspond à un écran de l'application ; les captures sont à déposer dans `figures/captures/` (voir README du dossier).

5.2 Espace public et connexion

5.2.1 Page d'accueil (landing)



[Capture : figures/captures/landing.png]

FIGURE 5.1 – Page d'accueil publique AmanaBank

5.2.2 Page de connexion

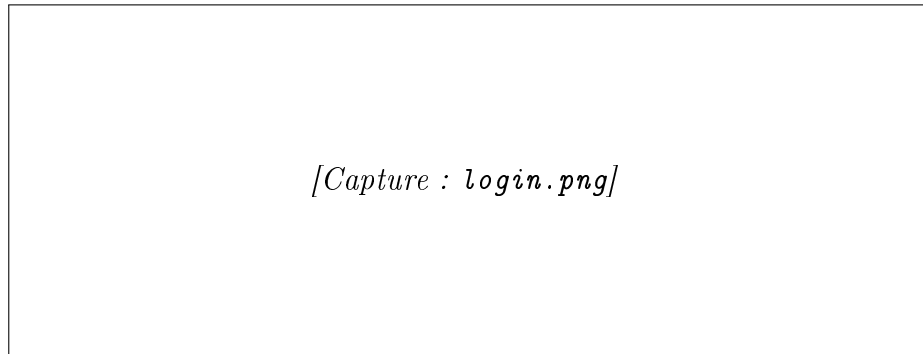


FIGURE 5.2 – Connexion personnel et client (CIN, n° client ou username)

5.3 Espace agence — personnel

5.3.1 Tableau de bord

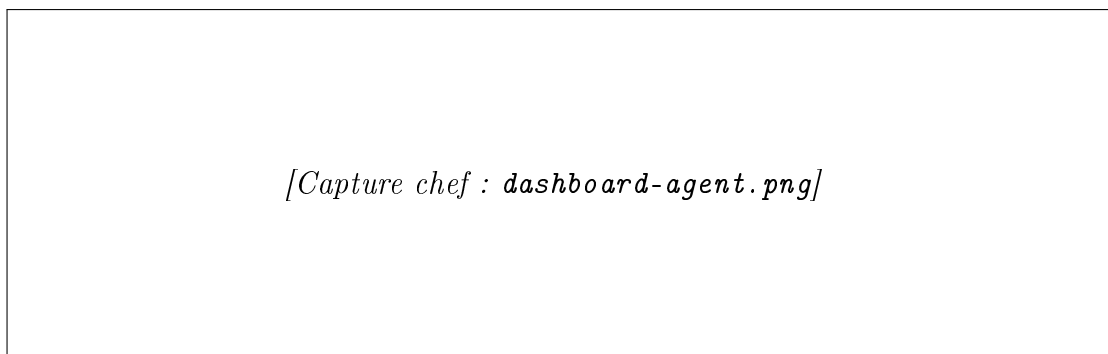


FIGURE 5.3 – Dashboard avec indicateurs et alertes chéquier

5.3.2 Gestion des clients

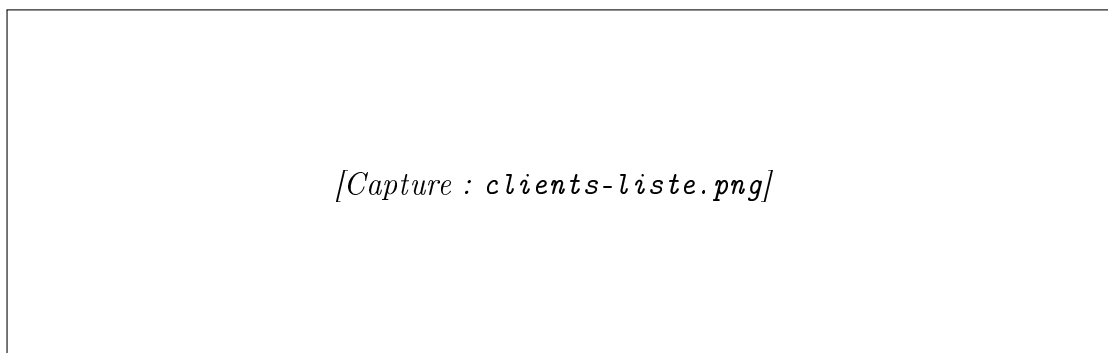


FIGURE 5.4 – Liste et recherche des clients

5.3.3 Comptes bancaires

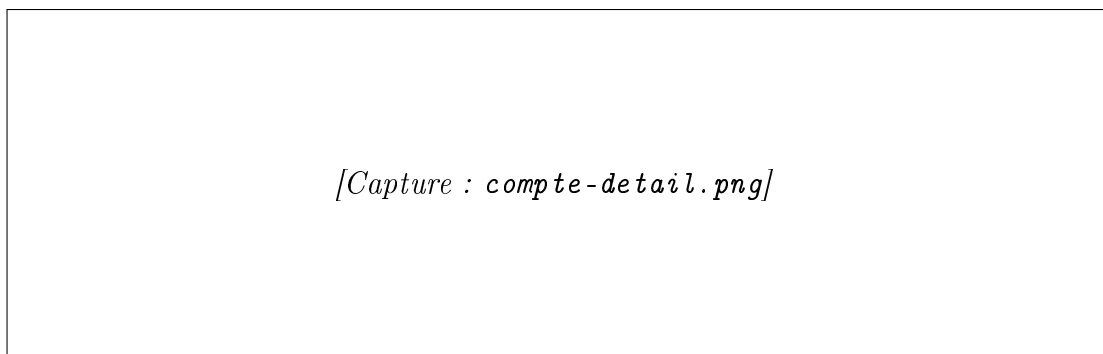


FIGURE 5.5 – Fiche détail compte — solde, statut, actions

5.3.4 Opérations financières

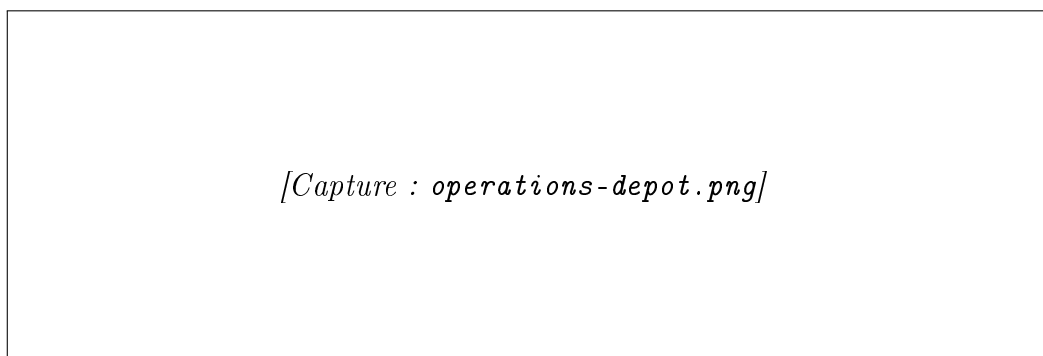


FIGURE 5.6 – Formulaire de dépôt

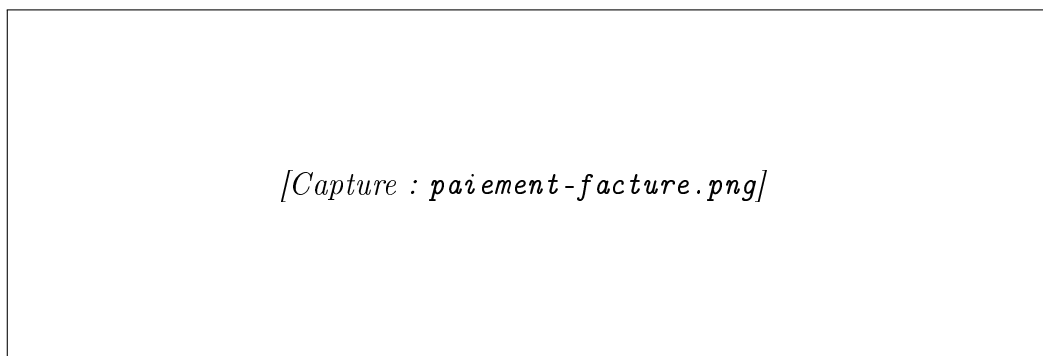


FIGURE 5.7 – Paiement de facture (LYDEC, ONEE, IAM...)

5.3.5 Historique et reçu PDF

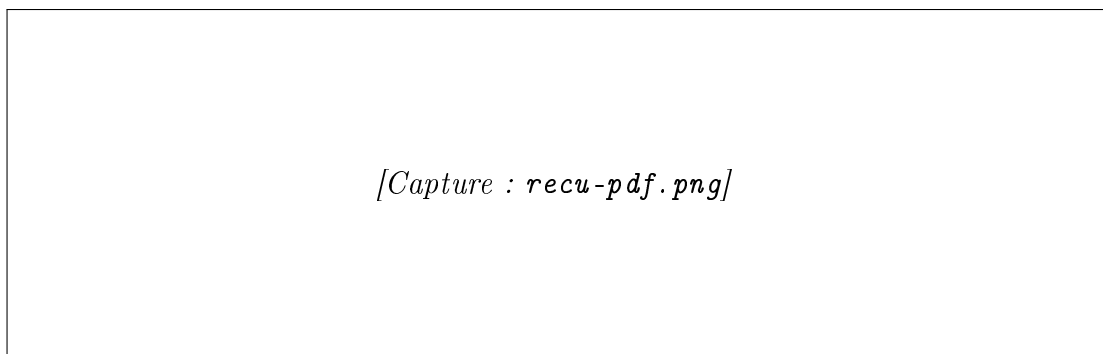


FIGURE 5.8 – Reçu d'opération avec téléchargement PDF

5.3.6 Commandes de chéquier et notifications

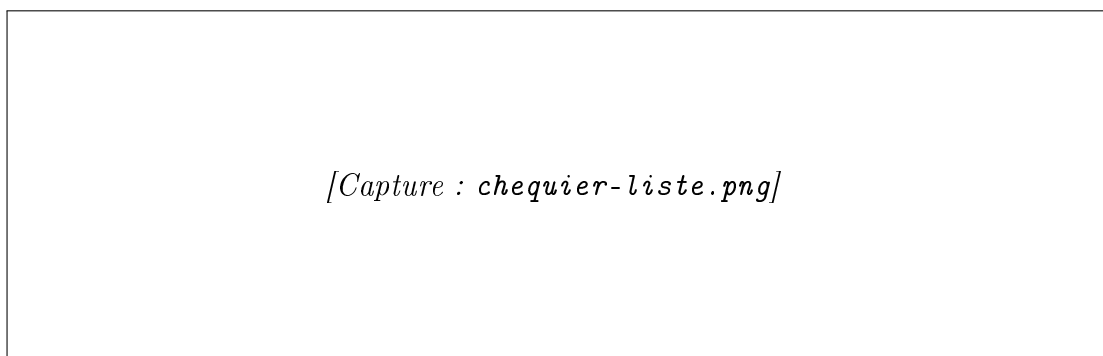


FIGURE 5.9 – Suivi des commandes de chéquier

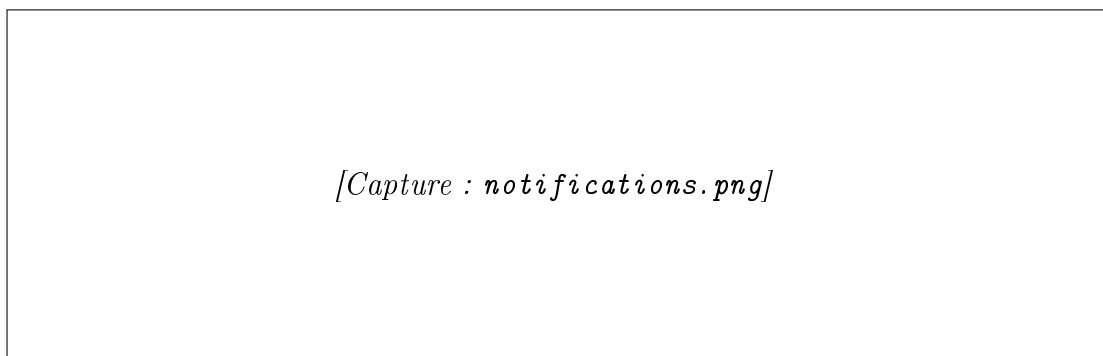


FIGURE 5.10 – Centre de notifications (chef d'agence)

5.3.7 Administration

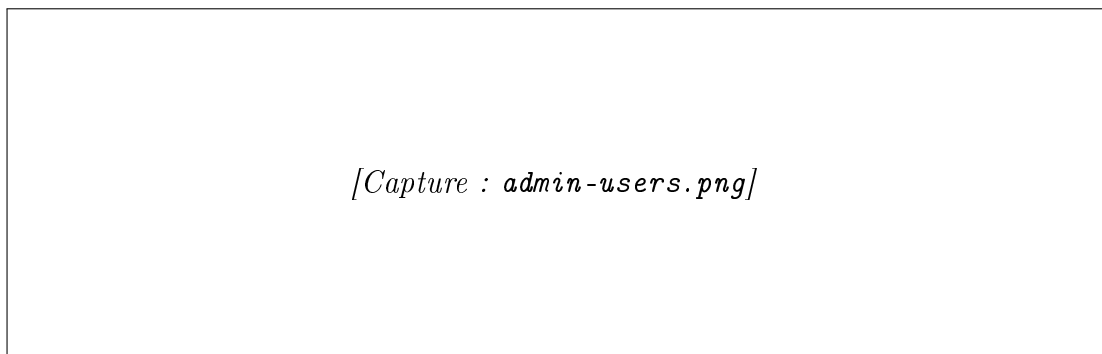


FIGURE 5.11 – Gestion des utilisateurs internes

5.4 Espace client (portail)

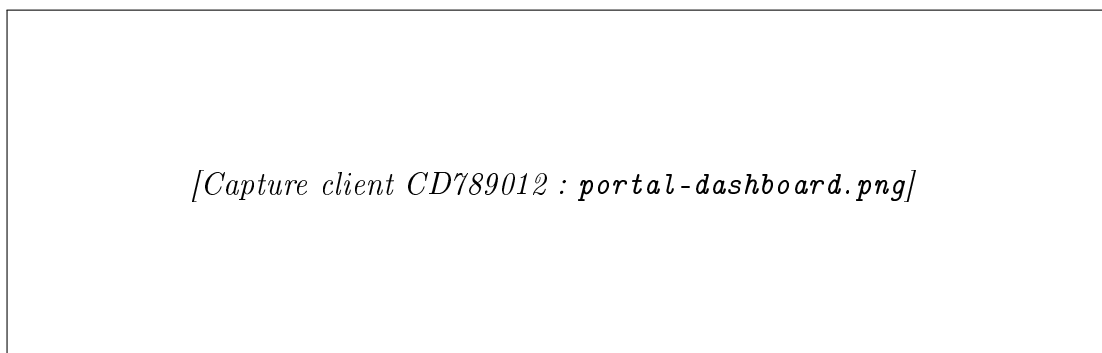


FIGURE 5.12 – Tableau de bord espace client

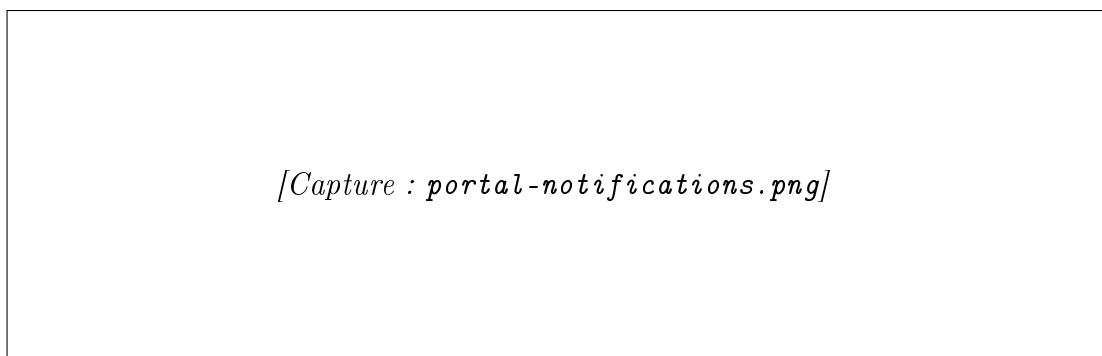


FIGURE 5.13 – Notifications client après opération en agence

5.5 Conclusion

Ce chapitre a illustré les principales interfaces de AmanaBank : vitrine publique, back-office agence et portail client. L'ensemble reflète une charte graphique cohérente et une

séparation claire des rôles.

Conclusion générale

Conclusion

La réalisation de ce projet de fin d'année, intitulé **AmanaBank**, nous a permis de mener un développement web complet, de la conception UML jusqu'à une application fonctionnelle et testée. Destinée à la gestion d'une agence bancaire, cette plateforme répond aux exigences du cahier des charges et intègre des extensions métier : paiement de factures, commande de chéquier, notifications et portail client.

Bilan

- **Conception** : diagrammes UML complets (cas d'utilisation, classes, séquences, activité, MCD/MLD).
- **Implémentation** : 11 migrations Flyway, 9 entités JPA, architecture en couches Spring Boot.
- **Fonctionnalités** : gestion clients/comptes, opérations financières contrôlées, admin, audit, reporting, reçus PDF.
- **Extensions** : factures, chéquier, notifications bidirectionnelles, portail client lecture seule.
- **Qualité** : 80 tests d'intégration, seed de démo modulaire, documentation technique complète.
- **Interface** : charte AmanaBank cohérente (landing, agence, portail).

Les objectifs fixés en introduction ont été atteints : centralisation des données, respect des règles métier, traçabilité et expérience utilisateur adaptée à chaque rôle.

Difficultés rencontrées

- **Configuration PostgreSQL** : conflit de ports locaux (5432 occupé) — résolu en utilisant le port 5433.
- **Cohérence transactionnelle** : gestion des soldes avec verrouillage optimiste (`@Version`) et `@Transactional`.
- **Authentification dual** : un seul formulaire de login pour personnel et clients — résolu via `UserDetailsServiceImpl` composite.

- **Portail sur base existante** : synchronisation des accès client via `DemoPortalSync` au démarrage.
- **Workflow chéquier** : distinguer service administratif (sans mouvement financier) des transactions.

Perspectives d'évolution

- Export PDF des **relevés de compte** (reçus déjà en PDF).
- API REST documentée (Swagger) pour intégrations futures.
- Pipeline CI/CD (GitHub Actions) et déploiement conteneurisé.
- Enrichissement du portail client (demandes en ligne, messagerie).
- Authentification à deux facteurs (2FA) pour le personnel.
- Tableau de bord analytique avancé (graphiques, exports Excel).

En conclusion, AmanaBank constitue une solution concrète et défendable pour la digitalisation d'une agence bancaire. Ce PFA nous a permis de consolider nos compétences en conception orientée objet, développement Java/Spring et travail en binôme, et constitue une base solide pour nos futurs projets professionnels.

BENSOUDA Amina
AFILAL Soraya

Bibliographie

- [1] Spring Boot Documentation, VMware / Broadcom. <https://docs.spring.io/spring-boot/docs/current/reference/html/> (consulté en 2026).
- [2] Spring Security Reference Documentation. <https://docs.spring.io/spring-security/reference/> (consulté en 2026).
- [3] Spring Data JPA Documentation. <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/> (consulté en 2026).
- [4] Thymeleaf Documentation. <https://www.thymeleaf.org/documentation.html> (consulté en 2026).
- [5] PostgreSQL Documentation, Version 18. <https://www.postgresql.org/docs/> (consulté en 2026).
- [6] Flyway Documentation, Redgate. <https://documentation.red-gate.com/fd> (consulté en 2026).
- [7] Bootstrap Documentation, Version 5.3. <https://getbootstrap.com/docs/5.3/> (consulté en 2026).
- [8] OpenPDF — Libre PDF Library. <https://github.com/LibrePDF/OpenPDF> (consulté en 2026).
- [9] OMG — Unified Modeling Language Specification. <https://www.omg.org/spec/UML/> (consulté en 2026).
- [10] PlantUML — Open-source tool for UML diagrams. <https://plantuml.com/> (consulté en 2026).
- [11] Cahier des charges PFA — Système de gestion d'agence bancaire. [documentation/cahier-charge-PFA.pdf](#), projet AmanaBank.
- [12] GanttProject — Project scheduling application. <https://www.ganttproject.biz/> (consulté en 2026).